

Variable Slices

In "Slice Literals," we already knew the exact data going in. But what if you need an empty slice to hold data that you are going to fetch later? For that, Go gives us two powerful functions: **make()** and **append()**.

1. Creating Empty Slices with make()

The make function explicitly creates a brand-new slice and its hidden underlying array in computer memory. You can call it in two ways:

The Two-Argument Version (Length = Capacity):

It creates an underlying array of size 10, filled with zeros. It then creates a slice with a **Length of 10** and a **Capacity of 10**.

```
slice := make([]int, 10)
```

The Three-Argument Version (Length < Capacity):

It creates a larger underlying array of size 20. However, it sets the slice's **Length to 0**, but its **Capacity to 20**.

```
slice := make([]int, 0, 20)
```

Why do this? It gives you an empty slice that is ready to accept data, but you have pre-reserved memory space (capacity), so the slice has plenty of room to grow without asking the computer for more RAM later.

2. Growing Slices with append()

Unlike arrays, you can constantly add new elements to the end of a slice using the **append()** function.

```
slice = append(slice, 100)
```

This takes the existing slice, adds the number 100 to the end, and returns the updated slice.

How it interacts with Length and Capacity:

- 1. When you have room:** If your slice has a **Length of 0** and a **Capacity of 3**, calling append will drop the new data into the underlying array and increase the **slice's Length to 1**. **The Capacity stays at 3**.
- 2. When you run out of room:** If your slice is full (**Length is 3, Capacity is 3**) and you try to append another item, Go does something magical behind the scenes.
 - It pauses.
 - It creates a brand-new, much larger underlying array in memory.
 - It copies all your old data into the new array.
 - It adds your new item.
 - It points your slice to this new array and throws the old array into the Garbage Collector.

Expert Takeaway:

In **JavaScript**, you can just **.push()** items into an array forever, and the engine handles it. Go's **append** works similarly, but with a **massive performance caveat**.

Every time **append** runs out of **capacity** and has to create a **brand-new underlying array**, it costs valuable **CPU cycles**. If you are writing a backend function that processes **10,000** JSON records, doing

```
sli := make([]int, 0)
```

Appending 10,000 times will constantly trigger that heavy background **copy process**. By using the three-argument **make** function to pre-allocate the capacity **make([]int, 0, 10000)**, your slice never runs out of room, avoiding the CPU penalty entirely.